

claude-windows / 7dc3da51-af38-47b6-83f6-fdda83b60664

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\subagents\workflows\wf_e1b943dd-dac\agent-ae589b655905103df.jsonl`  
- SHA-256: `52d0c34bf42a40af93e686cd137d26b56972320fd10d3af2342d38ac26776777`  
- Source modified: `2026-07-06T19:18:44+00:00`  
- Imported at: `2026-07-08T16:00:12+00:00`  
- project: `wf_e1b943dd-dac`  
- session_id: `7dc3da51-af38-47b6-83f6-fdda83b60664`
```

Transcript

1. user (2026-07-06T19:17:45.563Z)

Reviewing GitHub PR #40 of the repo at C:/Users/avidu/Projects/Telegram ? a DRAFT, DOCS-ONLY PR (branch docs/track-f-policy-pipeline).
It adds ONE file: docs/adr/0012-policy-pipeline.md (ADR 0012, status Proposed) ? the Track F design doc for a four-tier "policy pipeline" (Tier 1 metadata gates / Tier 2 topic preferences / Tier 3 content safety / Tier 4 deep inspection), a PolicyStage protocol + StageResult explanation trace, and 7 phased follow-up issues F1-F7.
No code changes. Read the ADR:
git -C C:/Users/avidu/Projects/Telegram show docs/track-f-policy-pipeline:docs/adr/0012-policy-pipeline.md

This is a DESIGN review, not a code review ? there is no gate to run. Judge the ADR as a design contract:
soundness, completeness, internal consistency, accuracy of its claims, and consistency with the codebase and the other ADRs (docs/adr/0001..0011 exist on main; read any you need via 'git -C C:/Users/avidu/Projects/Telegram show main:docs/adr/<file>').
Relevant current code (on main): src/tg_video_filter/models.py (FilterConfig, ContentItem, Source, Policy),
src/tg_video_filter/db.py (load_filter_config/save_filter_config shims, sources.watched gate should_ingest_source, delivery_attempts/record_delivery), src/tg_video_filter/telethon_client.py (the live handler: watched gate -> dedup -> FilterEngine.evaluate -> insert -> deliver). Track E (sources.watched opt-in gate) and Track C (delivery_attempts) just merged.

Ground rules:

- READ actual files, cite ADR section / file:line. You MAY run C:/Users/avidu/Projects/Telegram/.venv/Scripts/python.exe to check claims (e.g. does FilterConfig ignore/drop unknown JSON keys? does ContentItem carry caption text? is content_item.text populated?).
- This is a DRAFT asking for design feedback ? frame findings as design gaps/risks/inaccuracies, not merge-blockers.
 - Severity: high (a claim that is false, or a design gap that would break an F-phase / cause data loss), medium (unspecified interaction, inconsistency, missing dependency), low (wording, invalid example, minor).
- Report only issues in THIS ADR. No praise. Be concrete: what's wrong, why it matters for implementation.

ADVERSARIALLY VERIFY this design finding ? try to REFUTE it. Read the cited ADR section + any referenced ADR/code,
run C:/Users/avidu/Projects/Telegram/.venv/Scripts/python.exe checks if useful. CONFIRMED if it genuinely holds against THIS ADR (a real gap/inaccuracy/inconsistency).
REFUTED if the ADR actually addresses it / the claim is wrong / it's out of scope for a design doc. PLAUSIBLE if unsettled. Corrected severity (remember: draft design doc, so calibrate ? false claim/data-loss = high, gap = medium).
FINDING (claims): 'Extra fields are ignored' is presented as purely positive, but the existing save path erases the new config keys ? any /set silently wipes the entire pipeline config severity high | section Schema ? minimal changes / Consequences (Positive) ? 'backwards-compatible with the existing FilterConfig Pydantic model (extra fields are ignored)' summary: The same 'extra fields ignored' property that makes reads backwards-compatible also makes the load->save round-trip DESTRUCTIVE: load_filter_config parses config_json into FilterConfig (dropping topics/allowlist/security_checks/deep_inspect), and save_filter_config re-serializes via cfg.model_dump_json() ? so any command that saves config (e.g. /set) rewrites the policy row without the pipeline keys, permanently wiping them.
why_it_matters: This is a data-loss interaction the ADR omits entirely. As written, a user configures topics/allowlist/deep_inspect, then a single /set duration=... erases all of it. F1/F2/F3/F4 cannot be built on the FilterConfig shim as-is: the ADR must state that F1 has to extend BOTH the parse path (add fields to FilterConfig or introduce a superset PolicyConfig model) AND the save path, and that until then the extra keys are not merely 'ignored' but actively destroyed on the next save. Verified in db.py:1698-1725 and models.py FilterConfig.
evidence: db.load_filter_config (db.py:1698-1708) does FilterConfig.model_validate_json(policy.config_json). db.save_filter_config (db.py:1711-1725) does config_json=cfg.model_dump_json(). FilterConfig (models.py:299-380) declares no extra-field config, so pydantic drops unknown keys on validate AND they are absent from model_dump_json. Python check: parsing {min_duration_s, topics, allowlist, deep_inspect, security_checks} then model_dump_json yields only the 9 known keys ['delivery_mode', 'delivery_target', 'exclude_gif', 'exclude_nosound', 'exclude_round', 'max_duration_s', 'max_size_mb', 'min_duration_s', 'target_height'] ? topics/allowlist/deep_inspect/security_checks all gone.
rec: Reframe the 'extra fields ignored' bullet as a two-sided fact and add an explicit note under 'Schema ? minimal changes' that F1 must (a) add the new keys to a parseable model (extend FilterConfig, or a new PolicyConfig superset) and (b) update save_filter_config / the /set write path to preserve them, otherwise the config is silently erased on the next save. Consider calling out that 'no schema migration' does not mean 'no code change to persistence'.

2. assistant (2026-07-06T19:17:47.418Z)

I'll verify this finding by reading the ADR and the cited code.

3. user (2026-07-06T19:17:49.598Z)

ADR 0012: Policy pipeline ? explainable, tiered, user-customizable content gates

- **Status**: Proposed
- **Date**: 2026-07-06
- **Decision owner**: project owner
- **Tracking issue**: #31 (Track F of the provider-neutral umbrella, #25)
- **Depends on**: ADR 0011 (Track B ? domain schema, `policies` table)

Context

The system today evaluates every matched video against a single `FilterConfig` (per account, via `policies.config\_json`). The filter is **flat**: all rules run unconditionally, the output is a boolean (`matched`), and there is no way to express topic preferences, security policy, or why a specific video passed or failed.

Track F defines a **policy pipeline** ? an ordered sequence of stages that run before delivery ? so the system can express richer decisions, explain them, and defer expensive work until after cheap rejections.

What we have today (post-Track-E)

```
| Concept | Current state |
|---|---|
| `FilterConfig` / `Policy` | One JSON blob per account. Flat duration/resolution/size/exclude
rules. AND semantics. |
| `FilterEngine` | Evaluates all rules; returns `FilterResult(passed, reasons)`. |
| `policies` table | One row per account (is_default=1). `config_json` is the `FilterConfig`
blob. |
| Security / topic / deep-inspect | Nothing. Codec/keyword/MIME filters and ffmpeg were
deferred to v2 in the original plan. |
```

What we want

A **policy** should be a pipeline of ordered **stages**, each with a well-defined cost, inputs, and explainable output. The pipeline runs **after** dedup and **before** delivery. Users should be able to express topic preferences ("I like AI and philosophy videos"), security rules ("reject links from sketchy domains"), and view **why** a specific item was accepted or rejected.

Decision

Stage model ? four ordered tiers

The pipeline is split into four tiers, ordered by cost (cheapest first). Later stages only run if all earlier stages pass:

```
...
????????????????????????????????????????????????????????????
?                               Policy Pipeline                       ?
?                               ?                                     ?
? Tier 1 ? Metadata gates (always runs, zero I/O)                   ?
?   duration, resolution, size, codec, mime, GIF/round              ?
?   ? extends today's FilterEngine                                  ?
?                               ?                                     ?
? Tier 2 ? Topic preferences (opt-in, metadata-only)                ?
?   source allowlist, keyword match, channel topic tags            ?
?   ? user-expressed interests filter the stream                   ?
?                               ?                                     ?
? Tier 3 ? Content safety (opt-in, metadata + light I/O)            ?
?   URL/domain reputation, file metadata sanity,                   ?
?   caption text scanning                                           ?
?   ? cheap checks; no download                                     ?
?                               ?                                     ?
? Tier 4 ? Deep inspection (opt-in, heavy I/O)                      ?
?   ffmpeg, perceptual hash, downloaded content scan               ?
?   ? gated behind explicit user opt-in + budget                   ?
????????????????????????????????????????????????????????????
...
```

Stage interface ? `PolicyStage`

Each stage is a pure function (or async function for I/O stages) that takes a `PolicyContext` and returns a `StageResult`:

```
```python
@dataclass
class PolicyContext:
 """Immutable snapshot of what the stage can inspect."""
 content_item: ContentItem # from ADR 0011
 source: Source # the source it came from
 policy: Policy # the owning policy config
```
```

```

@dataclass
class StageResult:
    passed: bool
    reason: str # human-readable explanation
    stage_name: str # e.g. "duration-gate", "url-reputation"
    metadata: dict[str, object] = field(default_factory=dict) # for explainability UI

class PolicyStage(Protocol):
    """A single gate in the pipeline. Stateless; may be async for I/O stages."""
    async def evaluate(self, ctx: PolicyContext) -> StageResult: ...

```

The pipeline evaluates stages in order and **short-circuits** on the first failure (don't waste I/O on an already-rejected item). The accumulated `StageResult` list is the **explanation trace** ? every decision is auditable.

Topic preferences ? Tier 2

Topic preferences are stored as a JSON array in `policies.config_json` under a `topics` key:

```

```json
{
 "min_duration_s": 5,
 "topics": {
 "include": ["AI", "Philosophy", "Soccer"],
 "exclude": ["Politics", "Crypto"],
 "mode": "any" // "any" = match any include topic; "all" = match all
 }
}
```

```

Topic matching is **metadata-only** in v1: source title substring match, message caption keyword match, and (future) channel topic tags from Telegram's channel metadata. No ML classification in the design ? that's a separate issue.

Security stages ? Tier 3

Security stages are **opt-in** and configured per-policy. The design defines three initial stages (implementation deferred to separate issues):

1. **Source allowlist**: reject items from sources not in the `allowlist` (an explicit list of trusted `provider_source_ids`).
2. **URL/domain reputation**: check links in captions against a configurable blocklist. Metadata-only ? no page fetch.
3. **Duplicate suppression**: enhanced dedup that compares by `content_fingerprint` across accounts (already handled by `content_items.dedup_key`; this stage adds cross-policy duplicate detection within an account's delivery streams).

Each security stage records its decision in `StageResult.metadata` for explainability (e.g. `{"blocked_domain": "sketchy-site.com"}`).

Deep inspection ? Tier 4

Deep inspection (ffprobe, perceptual hash, downloaded content scan) is **gated** behind explicit opt-in because it violates ADR 0003's no-download principle. The opt-in is per-policy (`deep_inspect: true` in `config_json`) and the stage has a **budget** (max bytes per item, max concurrent downloads) enforced by the policy engine.

This tier is explicitly deferred to a follow-up issue ? the design only

****reserves the slot**** in the pipeline model so it doesn't require a schema migration when implemented.

Policy evaluation flow (post-dedup, pre-delivery)

```
```python
async def evaluate_pipeline(
 ctx: PolicyContext,
 stages: list[PolicyStage],
) -> tuple[bool, list[StageResult]]:
 results: list[StageResult] = []
 for stage in stages:
 result = await stage.evaluate(ctx)
 results.append(result)
 if not result.passed:
 return False, results # short-circuit
 return True, results
```
```

The caller (telethon_client or future Bot API handler) records the results via `delivery\_attempts.last\_error` (for failed items) or a new `policy\_decisions` table (for explainability ? deferred to implementation).

Schema ? minimal changes

****No schema migration**** is required for this design. The `policies` table already holds a `config\_json` JSON blob. The pipeline is a code concept:

- `policies.config\_json` gains optional `topics`, `allowlist`, `security\_checks`, `deep\_inspect` keys ? all backwards-compatible with the existing `FilterConfig` Pydantic model (extra fields are ignored).
- Future `policy\_decisions` table (deferred) would store the explanation trace for debugging: `(content\_item\_id, policy\_id, stage\_name, passed, reason, metadata\_json, evaluated\_at)`.

Implementation phasing (future issues)

| Phase | Scope | Dependencies |
|--------|---------------------------|--|
| --- | --- | --- |
| **F1** | ? Policy engine prototype | `PolicyStage` protocol, `evaluate_pipeline()`, wire into live-message handler None |
| **F2** | ? Topic preferences | `topics` config parsing, source title/caption keyword matching F1 |
| **F3** | ? Source allowlist | `allowlist` config, skip non-allowlisted sources F1 |
| **F4** | ? URL/domain reputation | Domain blocklist, caption link extraction F1 |
| **F5** | ? Deep inspection | ffprobe/perceptual hash, download budget enforcement F1, ADR 0003 waiver |
| **F6** | ? Policy decisions table | Audit trail storage, explainability UI F1 |
| **F7** | ? Explainability endpoint | `/why <content_item_id>` command showing the explanation trace F6 |

Each phase is a separate, mergeable PR ? none are bundled here. This ADR only defines the ****design contract**** the phases code against.

Consequences

****Positive****

- Unifies topic, security, and quality gates into a single explainable framework ? no ad-hoc if-chains scattered across handlers.
- Cheap gates always run first; expensive enrichment is only reached for items that pass the cheap gates.
- `StageResult` list provides an audit trail (explainability) without additional DB lookups at decision time.
- Backwards-compatible: `policies.config\_json` already exists; adding

- optional keys requires no migration.
- Each implementation phase is independently shippable behind feature flags.

****Negative****

- Four tiers add conceptual complexity vs. today's flat filter.
- `PolicyStage` protocol requires all stages to conform to the same interface, which may be awkward for deeply different checks (metadata-only vs. I/O-heavy). Mitigated by the `metadata` dict catch-all.
- Topic matching is string-based substring matching in Phase 1 ? not semantic. A future ML stage would replace it, but it's good enough for a v1.
- Deep inspection violates ADR 0003 and must be explicitly opt-in with a budget to avoid runaway costs.

Alternatives considered

- ****Do nothing; keep flat FilterConfig.**** Rejected ? the umbrella exists to dissolve the conflation of quality, topic, and security into a single boolean. A pipeline is the natural generalization.
- ****Inline each stage in the handler**** (no protocol). Rejected ? makes it impossible to test stages independently, reorder them, or add new ones without touching the hot path.
- ****Use a rules engine / DSL.**** Rejected ? over-engineered for <100 users. A Python list of callables is simpler, testable, and introspectable.
- ****Bundle all phases into a single PR.**** Rejected ? scope discipline. Each phase is independently shippable; this ADR defines the shared interface they code against.

Open questions for the decision owner

1. ****`StageResult.metadata` shape**** ? dict or typed per-stage? Recommend: dict for Phase 1 (flexible), typed in Phase 2 (once the shape stabilizes).
2. ****Policy decisions table**** ? store every decision, or only failures? Recommend: every decision (explainability), with a TTL-based cleanup for old traces.
3. ****Topic matching**** ? substring only, or regex/glob? Recommend: substring (case-insensitive) for v1; natural-language matching in a follow-up.
4. ****Deep inspection budget**** ? per-item byte limit or per-account bandwidth cap? Recommend: per-item limit (10 MB default) with a per-account monthly cap enforced at the app level.

References

- ADR 0003 ? metadata-first probing (the no-download principle that Tier 4 must opt in to violate).
- ADR 0009 ? selective channel scanning (the `topics.allowlist` concept generalizes `sources.watched`).
- ADR 0011 ? provider-neutral domain schema (the `policies` table this design extends, `ContentItem` as the `PolicyContext` input).
- Issue #31 ? Track F (this ADR's tracking issue).
- Issue #25 ? umbrella; this is the final track.

4. user (2026-07-06T19:17:50.268Z)

```

1690         forwarded=False,
1691         seen_at=item.seen_at,
1692     )
1693 
```

```

1694
1695 # --- filter config shims (backwards-compatible view over policies) -----
1696
1697
1698 async def load_filter_config(conn: aiomysqlite.Connection, user_id: int) -> FilterConfig:
1699     """Deprecated shim: load the account's default :class:`Policy` as FilterConfig.
1700
1701     ADR 0011 (Option 2A): ``filter_config`` is renamed to
1702     ``filter_config_deprecated``; this reads ``policies WHERE is_default=1``
1703     instead. Falls back to :data:`FilterConfig` defaults if no policy exists.
1704     """
1705     policy = await get_default_policy(conn, user_id)
1706     if policy is None:
1707         return FilterConfig()
1708     return FilterConfig.model_validate_json(policy.config_json)
1709
1710
1711 async def save_filter_config(conn: aiomysqlite.Connection, user_id: int, cfg:
FilterConfig) -> None:
1712     """Deprecated shim: upsert the account's default :class:`Policy`."""
1713     existing = await get_default_policy(conn, user_id)
1714     await upsert_policy(
1715         conn,
1716         Policy(
1717             id=existing.id if existing is not None else None,
1718             account_id=user_id,
1719             name="default",
1720             config_json=cfg.model_dump_json(),
1721             is_default=True,
1722             created_at=(existing.created_at if existing is not None else _now()),
1723             updated_at=_now(),
1724         ),
1725     )
1726
1727
1728 # --- bulk helpers (used by the manager on startup) -----
1729
1730
1731 async def list_videos_for_user( # v2: used by on-demand /search; not called in v1
1732     conn: aiomysqlite.Connection, user_id: int, limit: int = 100
1733 ) -> list[VideoRecord]:
1734     """Deprecated shim: list the account's content items projected onto VideoRecord."""
1735     conn.row_factory = aiomysqlite.Row
1736     async with conn.execute(
1737         """SELECT ci.*, s.provider_source_id AS src_pid
1738            FROM content_items ci
1739            JOIN sources s ON s.id = ci.source_id

```

5. assistant (2026-07-06T19:17:52.719Z)

The finding hinges on whether `FilterConfig.model_dump_json()` drops unknown keys. Let me verify the model definition and test the round-trip empirically.

6. user (2026-07-06T19:17:53.840Z)

```

295     delivered_at: datetime | None = None
296     next_retry_at: datetime | None = None
297
298
299 class FilterConfig(BaseModel):
300     """Per-user filter configuration.
301
302     v1 rules: duration bounds, minimum height (resolution), and max file size.
303     Stored in the DB as a JSON blob (``filter_config.config_json``) so adding a
304     new field later is backwards-compatible.

```

```

305     """
306
307     # Duration bounds in seconds. None means unbounded on that side.
308     min_duration_s: int | None = Field(default=5, ge=0)
309     max_duration_s: int | None = Field(default=7200, ge=0)
310
311     # Resolution: a video passes if its height >= target_height.
312     # None disables the resolution rule.
313     target_height: int | None = Field(default=1080, ge=1)
314
315     # Max file size in mebibytes. None disables the size rule.
316     max_size_mb: int | None = Field(default=2048, ge=1)
317
318     # Content-type exclusions. GIFs and round videos are excluded by default
319     # (their duration/dimensions are usually incidental to real video filters);
320     # silent videos are off by default since some legit clips have no audio.
321     # Existing users' stored configs are unaffected ? the DB JSON blob overrides
322     # these defaults, so this only changes the behaviour for fresh installs.
323     exclude_gif: bool = Field(
324         default=True,
325         description="If True, exclude Telegram GIFs (DocumentAttributeAnimated)",
326     )
327     exclude_round: bool = Field(
328         default=True,
329         description="If True, exclude circular video messages (round_message=True)",
330     )
331     exclude_nosound: bool = Field(
332         default=False,
333         description="If True, exclude videos without an audio track (nosound=True)",
334     )
335
336     # How matched videos are delivered: forward the video, send a link, or both.
337     # Stored in the same JSON blob, so adding this field is backwards-compatible
338     # (existing rows without it default to FORWARD).
339     delivery_mode: DeliveryMode = Field(default=DeliveryMode.FORWARD)
340
341     # Where matched videos / link messages are sent. Accepts:
342     #   - "me" (default): Saved Messages ? exact v1.1 behaviour.
343     #   - "@username": a public channel the account is a member of.
344     #   - "-100..." or a bare int-as-string: a numeric Telegram chat id
345     #     (private channels without a username).
346     # The string is passed verbatim to Telethon's forward_messages / send_message.
347     # See ADR 0005 for rationale and access requirements.
348     delivery_target: str = Field(
349         default="me",
350         min_length=1,
351         description="Telethon entity to deliver to: 'me', '@username', or a chat id",
352     )
353
354     @field_validator("delivery_target", mode="before")
355     @classmethod
356     def _coerce_delivery_target(cls, v: object) -> str:
357         """Accept int chat ids (e.g. -1001234567890) and coerce to str.
358
359         Telethon accepts both str and int as an entity, but storing as str keeps
360         the JSON blob uniform and lets '@username' / 'me' / numeric ids share
361         one field type.
362         """
363         if v is None:
364             return "me"
365         if isinstance(v, int):
366             return str(v)
367         return str(v)
368
369     @model_validator(mode="after")

```



```

370     def _check_duration_bounds(self) -> FilterConfig:
371         """Ensure min_duration_s <= max_duration_s when both are set."""
372         if (
373             self.min_duration_s is not None
374             and self.max_duration_s is not None
375             and self.min_duration_s > self.max_duration_s
376         ):
377             raise ValueError(
378                 f"min_duration_s ({self.min_duration_s}) must be <= max_duration_s
379                 ({self.max_duration_s})"
380             )
381         return self
382
383     class VideoRecord(BaseModel):
384         """A seen video message + its metadata. Drives dedup and future /search.

```

7. assistant (2026-07-06T19:18:00.090Z)

No `model\_config = ConfigDict(extra=...)`, so pydantic default is `extra="ignore"` ? unknown keys are dropped on validation and absent from `model\_dump\_json`. Let me confirm empirically and check what `/set` actually calls.

8. user (2026-07-06T19:18:02.852Z)

```

keys after round-trip: ['delivery_mode', 'delivery_target', 'exclude_gif', 'exclude_nosound',
'exclude_round', 'max_duration_s', 'max_size_mb', 'min_duration_s', 'target_height']
topics present? False
allowlist present? False
deep_inspect present? False
security_checks present? False
model_config extra: None

```

9. user (2026-07-06T19:18:03.051Z)

```

src\tg_video_filter\models.py:193:     ``load_filter_config`` / ``save_filter_config`` shims
read/write that row.
src\tg_video_filter\manager.py:140:         cfg = await db.load_filter_config(conn,
session.account_id)
src\tg_video_filter\command_service.py:465:         await db.save_filter_config(conn, user_id,
cfg)
src\tg_video_filter\bootstrap.py:75:         await db.save_filter_config(conn, user.id,
DEFAULT_FILTER_CONFIG)
src\tg_video_filter\commands.py:78:         current_cfg = await db.load_filter_config(conn,
user.id)
src\tg_video_filter\commands.py:82:         new_cfg = await db.load_filter_config(conn,
user.id)
src\tg_video_filter\commands.py:112:     cfg = await db.load_filter_config(conn, user_id)
src\tg_video_filter\db.py:618:     # (Option 2A). is_default=1 so the legacy
load_filter_config shim reads
src\tg_video_filter\db.py:1698:async def load_filter_config(conn: aiomysqlite.Connection, user_id:
int) -> FilterConfig:
src\tg_video_filter\db.py:1711:async def save_filter_config(conn: aiomysqlite.Connection, user_id:
int, cfg: FilterConfig) -> None:
src\tg_video_filter\db.py:1809:     "load_filter_config",
src\tg_video_filter\db.py:1810:     "save_filter_config",

```

10. assistant (2026-07-06T19:18:08.060Z)

The round-trip confirms all four pipeline keys are dropped. Now let me verify the `/set` write path actually goes load ? mutate ? save (the destructive round-trip).

11. user (2026-07-06T19:18:08.601Z)

```

440         max_size_mb: int | None = None,
441         max_duration_s: int | None = None,
442         min_duration_s: int | None = None,
443         target_height: int | None = None,
444         exclude_gif: bool | None = None,
445         exclude_round: bool | None = None,
446         exclude_nosound: bool | None = None,
447     ) -> FilterConfig:
448         """Apply partial updates to the account's config, persist, and return."""
449         updates: dict[str, object] = {}
450         if max_size_mb is not None:
451             updates["max_size_mb"] = max_size_mb
452         if max_duration_s is not None:
453             updates["max_duration_s"] = max_duration_s
454         if min_duration_s is not None:
455             updates["min_duration_s"] = min_duration_s
456         if target_height is not None:
457             updates["target_height"] = target_height
458         if exclude_gif is not None:
459             updates["exclude_gif"] = exclude_gif
460         if exclude_round is not None:
461             updates["exclude_round"] = exclude_round
462         if exclude_nosound is not None:
463             updates["exclude_nosound"] = exclude_nosound
464         cfg = current.model_copy(update=updates) if updates else current
465         await db.save_filter_config(conn, user_id, cfg)
466         log.info(
467             "config_updated user=%s max_size_mb=%s min_duration_s=%s max_duration_s=%s "
468             "target_height=%s exclude_gif=%s exclude_round=%s exclude_nosound=%s",
469             user_id,

```

12. assistant (2026-07-06T19:18:11.114Z)

Let me confirm where `current` comes from ? it must be a loaded FilterConfig (already stripped) or freshly parsed.

13. user (2026-07-06T19:18:11.873Z)

```

425         # Try by title (case-insensitive substring match).
426         all_srcs = await db.list_account_sources(conn, user_id)
427         target_lower = target.lower()
428         for s in all_srcs:
429             if s.title and target_lower in s.title.lower():
430                 return s
431         return None
432
433     # -- internal -----
434
435     async def _update(
436         self,
437         conn: aiosqlite.Connection,
438         user_id: int,
439         current: FilterConfig,

```

14. user (2026-07-06T19:18:15.187Z)

```

228-         if new_mb < 1:
229-             return CommandResult("? Size must be at least 1 MB (`FilterConfig`
constraint).")
230:         cfg = await self._update(conn, user_id, current_cfg, max_size_mb=new_mb)
231-         --
232-         f">= min duration ({_format_duration(min_secs)})."
233-         )
234-         cfg = await self._update(conn, user_id, current_cfg, max_duration_s=new_secs)

```

```

--
268-         f"<= max duration ({_format_duration(max_secs)})."
269-     )
270-     cfg = await self._update(conn, user_id, current_cfg, min_duration_s=new_min)
--
280-     if new_height < 1:
281-         return CommandResult("? Resolution must be at least 1p.")
282-     cfg = await self._update(conn, user_id, current_cfg, target_height=new_height)
--
303-         "nosound": "exclude_nosound",
304-     }
305-     cfg = await self._update(conn, user_id, current_cfg, **{field_map[what]:
enabled})
--
433-     # -- internal -----
434-
435-     async def _update(

```

15. assistant (2026-07-06T19:18:28.097Z)

`current\_cfg` is loaded via `db.load\_filter\_config` (already confirmed via grep at commands.py and the service). The full destructive chain is confirmed: `load\_filter\_config` (parses to FilterConfig, dropping pipeline keys) ? `model\_copy` ? `save\_filter\_config` (`model\_dump\_json`, omitting pipeline keys). Any `/set` command permanently erases `topics`/`allowlist`/`security\_checks`/`deep\_inspect`.

Now let me assess whether the ADR actually addresses this. The finding claims the ADR presents "extra fields are ignored" as purely positive and omits the save-path destruction. Reading the ADR's "Schema ? minimal changes" and F1 phase description confirms:

- "extra fields are ignored" is stated only as a positive (backwards-compat for reads).
- F1's scope is "`PolicyStage` protocol, `evaluate\_pipeline()`", wire into live-message handler" with **dependencies: None** ? it does not mention extending FilterConfig's parse path or the save path.
- The ADR explicitly says "No schema migration is required" and the pipeline is "a code concept," but never notes that persistence code (`save\_filter\_config` / `\_update`) must change to preserve the new keys.

The finding holds. One nuance to check for adversarial fairness: F2/F3/F4 depend on F1 and would presumably add config parsing ? but the ADR never states this, and critically the coexistence problem remains. Even after F2 adds `topics` to a model, if the `/set` path still round-trips through a model lacking `allowlist`/`deep\_inspect` (added in later phases), those keys are wiped. The ADR's "extra fields are ignored" framing actively invites the destructive interpretation. This is a real, unaddressed gap.

On severity: the ADR is a design contract for phases that will be built on this exact shim. If F1/F2 land without the persistence-path fix the ADR omits, a user's pipeline config is silently and permanently erased on the next `/set` ? data loss. The "extra fields are ignored" bullet is also inaccurate as a one-sided positive. High is justified for a design doc whose omission would cause data loss in the phases it governs.

16. user (2026-07-06T19:18:44.487Z)

Structured output provided successfully